TASK 4 – Trace the Compilation Process (4 Marks)

File: Task04/billing.c

```
#include <stdio.h>

int main(void)
{
    unsigned int units = 250;
    long long bill_paise = 3000LL + 100LL * 225LL + 50LL * 450LL;

    printf("Units: %u\\n", units);
    printf("Bill: Rs %lld.%02lld\\n", bill_paise / 100, bill_paise % 100);
    return 0;
}
```

GCC commands

gcc --version
gcc -std=c11 -Wall -Wextra -pedantic -E billing.c -o billing.i
gcc -std=c11 -Wall -Wextra -pedantic -S billing.c -o billing.s
gcc -std=c11 -Wall -Wextra -pedantic -c billing.c -o billing.o
gcc billing.o -o billing
./billing

gcc -E: preprocessor output. Header inclusions and macros are expanded; preprocessing directives are handled.

gcc -S: compiler output in assembly language, billing.s.

gcc -c: creates object code, billing.o, without linking an executable.

gcc billing.o -o billing: links the object file and libraries to create the executable.

Compile-time error: a syntax error such as a missing semicolon. Link-time error: calling a declared function whose definition is not linked. Run-time/input error: a user enters letters instead of a number; validate scanf's return value.

Warnings from -Wall -Wextra -pedantic identify suspicious code such as mismatched format specifiers, unused variables, or non-standard constructs. Fix warnings instead of ignoring them.

Single-file versus multi-file build

For a multi-file design (input.c, tariff.c, report.c plus a shared header), compile each source separately with gcc -c and link the resulting object files. After a change to one .c file, normally only that file needs recompilation followed by relinking. In a single-file design, the whole source is recompiled after a change. Object files use additional disk space, but incremental builds can save time for larger projects. The final program still processes n bills in O(n) time; each individual bill calculation is O(1).

Smart-Meter Design Challenge

The original design should be modified, not discarded. Keep integer-paise arithmetic, input validation, modular meter-rollover handling, and centralized tariff definitions. Replace the simple monthly reading input with timestamped 15-minute interval records. Store or stream the interval readings, classify each interval using the time-of-day tariff table, and calculate each charge using the tariff effective at that time. Keep tariff rates in one configuration/table rather than copying constants throughout the program. Validate missing, duplicate, out-of-order, or implausible readings. If m intervals are processed in chronological order, processing can be O(m); streaming needs O(1) working space beyond the current interval and tariff data, while storing all intervals needs O(m) space.

Testing Checklist

Normal usage: previous 1000, current 1250 → 250 units and Rs 480.00 bill under the stated example tariff.

Rollover: previous 65530, current 5 → 11 units consumed.

Invalid input: enter letters at a numeric prompt → program displays an error and exits safely.

DEBUG check: compile once normally and once with -DDEBUG; intermediate values should appear only in the debug build.

Compile all programs with -Wall -Wextra -pedantic and resolve warnings.